Q30: Inside the Framework: the PyTorch Compiler and Autodiff Memory

DLS 2026 — module author: Kukla

October 2026

Learning objectives

- Explain the layers of the `torch.compile` stack and what each does.
- Contrast tape-based (reverse-mode) and functional autodiff and their memory behavior.
- Reason about the activation-memory cost of training and how rematerialization trades it for compute.

Companion videos (watch in order):

- *What is Automatic Differentiation?* — Ari Seff (14:24). https://www.youtube.com/watch?v=wG_nF1awSSY
- *torch.compile: The Missing Manual* — PyTorch (15:15). https://www.youtube.com/watch?v=rew5CSUaIXg

1 The question

Student response #29 asks how the PyTorch compiler works and what its layers are, and why functional autodiff can use far more memory than tape-based autodiff. These are two of the most "internals" questions in the whole survey, and both reward a precise, mechanism-level answer.

2 Eager vs. compiled execution

By default PyTorch runs **eagerly**: each operation dispatches and executes immediately, which is easy to debug but leaves performance on the table (Python overhead, no cross-op fusion). `torch.compile` (introduced in PyTorch 2.0) **just-in-time compiles** the model to recover that performance while keeping eager semantics [1]. It does this in a stack of distinct layers.

3 The torch.compile stack, layer by layer

- **TorchDynamo** hooks CPython's frame evaluation to *trace* Python bytecode into an FX graph. When it hits something it cannot trace (a data-dependent branch, an unsupported call) it inserts a **graph break**, running that part eagerly and resuming tracing after—so compilation degrades gracefully rather than failing.

- **AOTAutograd** traces *both* the forward and the backward graph ahead of time, so the compiler can optimize training, not just inference, and can decide jointly what to save vs. recompute.
- **A backend compiler**, by default **TorchInductor**, lowers the graph to fast kernels: it performs **operator fusion** (combining elementwise ops to cut memory traffic) and generates **Triton** kernels for the GPU (and C++/OpenMP for CPU).
- **Triton** is the GPU kernel language Inductor emits—a Python-like DSL that compiles to efficient GPU code without hand-writing CUDA.

The design philosophy is "capture what you can, fall back where you must," which is why `torch.compile` works on real models with messy Python.

4 Two flavors of autodiff

Automatic differentiation comes in two main styles. **Reverse-mode / tape-based** autodiff (classic PyTorch `autograd`) runs the forward pass while recording a *tape* of operations and their saved inputs, then walks the tape backward to accumulate gradients. **Functional** autodiff (JAX-style, `functorch`/`torch.func`) instead transforms a pure function into a new function that computes gradients, composing transformations (`grad`, `vmap`, `jit`) functionally.

5 Why functional AD can use more memory

The crux of the student's question. Reverse-mode AD must keep the intermediate **activations** needed by the backward pass alive from the forward pass—this activation memory, not the parameters, usually dominates training memory, and it grows with network depth and batch size. A naive *functional* formulation can be *worse*: without the careful liveness analysis and in-place bookkeeping that a mature tape implementation does, composed transforms may materialize and retain more intermediate tensors (e.g. holding entire intermediate structures a transform could in principle free), inflating peak memory. The practical fix in both worlds is **gradient checkpointing / rematerialization**: save only a subset of activations and *recompute* the rest during the backward pass, trading extra compute for a large memory reduction—often turning an out-of-memory model into a trainable one at the cost of a second partial forward. The companion code (`code/autodiff_memory.py`) measures saved-activation count with and without checkpointing to make the trade concrete.

6 Why compile at all: a worked example of fusion

To see why the compiler stack earns its complexity, consider a trivial sequence: `y = relu(x @ W + b)`. In eager mode this is at least three separate GPU kernel launches (matmul, bias-add, relu), and the intermediate results are written to and read back from GPU memory between each—memory traffic, not arithmetic, is often the bottleneck. TorchInductor can **fuse** the bias-add and relu (and sometimes more) into a single generated Triton kernel, so the intermediate never leaves fast on-chip memory and the Python dispatch overhead disappears. Multiply that saving across every layer of a deep network and the speedup is substantial—frequently 1.3–2× on real training, more on inference—for a one-line change (`model = torch.compile(model)`). The graph-break mechanism is what makes this safe on real code: when Dynamo meets a construct it cannot trace, it compiles the traceable regions and runs the rest eagerly, so you get most of the speedup without having to rewrite your model to be "compiler-friendly."

7 The memory picture in one sentence

Tie the two halves together: the compiler (via AOTAutograd) sees the forward and backward graphs jointly, so it can reason about exactly which activations are needed for the backward pass and even apply rematerialization automatically—meaning the "why does my training run out of memory" question (dominated by saved activations) and the "how does the compiler help" question are, at the deepest level, the same question about the forward–backward activation lifetime.

8 Summary

`torch.compile` is a layered JIT: Dynamo traces bytecode (with graph breaks), AOTAutograd captures forward+backward, Inductor fuses and emits Triton/C++ kernels. Autodiff is reverse-mode/tape-based (records a tape, walks it back) or functional (transforms pure functions). Activation memory dominates training; a naive functional formulation can retain more intermediates than a tuned tape, and checkpointing trades recompute for memory to tame it.

References

[1] Jason Ansel, Edward Yang, Horace He, et al. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *ASPLOS*, 2024.